DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 – CO3

Mode: Pair Work – **2 Students per Group Duration:**
1 Week

Total Marks: 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

NAME:ALURI POOJITHA

ROLL.NO:192511230

Q1 – Decomposition and Elimination in Ordered Data (6 Marks)

Part A – Finding Maximum and Minimum

Shared group work

Problem. Given an array A of n elements, find the smallest and the largest element. Finding them independently needs $(n - 1) + (n - 1) = 2(n - 1)$ comparisons. The divide-and-conquer (tournament) strategy shares comparisons between the two searches.

- **Divide:** split A[low..high] at mid = $\lfloor (low + high)/2 \rfloor$ into a left half and a right half.
- **Base cases:** one element → it is both minimum and maximum (0 comparisons); two elements → one comparison decides which is which.
- **Each subproblem returns** the pair (minimum, maximum) of its own subarray.
- **Combine:** minimum = smaller of the two minima; maximum = larger of the two maxima (2 comparisons).

Pseudocode (DAA handbook format)

```
ALGORITHM MaxMin(A, low, high)
INPUT:
 A – array of elements; low, high – boundaries of the current subarray
OUTPUT:
 (minimum, maximum) of A[low..high]
BEGIN
 IF low = high THEN                    // base case: one element
  RETURN (A[low], A[low])
 END IF
 IF high = low + 1 THEN                // base case: two elements
  IF A[low] < A[high] THEN
   RETURN (A[low], A[high])
  ELSE
   RETURN (A[high], A[low])
  END IF
 END IF
 mid ← FLOOR((low + high) / 2)
 (min1, max1) ← MaxMin(A, low, mid)      // recursive call on left half
 (min2, max2) ← MaxMin(A, mid + 1, high)  // recursive call on right half
```

```
  IF min1 < min2 THEN minimum ← min1 ELSE minimum ← min2 END IF
  IF max1 > max2 THEN maximum ← max1 ELSE maximum ← max2 END IF
  RETURN (minimum, maximum)              // combine uses 2 comparisons
END
```

Manual trace (desk check) for A = [4, 9, 2, 7]

Step	Call	Action	Returns	Comparisons
1	MaxMin(0,3)	mid = 1 → call left and right	—	—
2	MaxMin(0,1)	two elements: 4 < 9	(4, 9)	1
3	MaxMin(2,3)	two elements: 2 < 7	(2, 7)	1
4	Combine	min(4,2) = 2 ; max(9,7) = 9	(2, 9)	2
		Total		**4 = 3·4/2 − 2**

Student A – Design and mathematical analysis

Decomposition strategy (n = 8). Every level halves the subarrays until pairs remain; the pairs are compared once and results are merged upward like a knockout tournament.

```
            [0..7]              level 0: combine = 2 comparisons
           /      \
      [0..3]      [4..7]        level 1: 2 combines × 2 = 4
      /   \       /   \
  [0..1] [2..3] [4..5] [6..7]   level 2: 4 pairs × 1 = 4 (base case)
```

How subproblems are formed: the two halves are independent – neither needs information from the other – and each is a smaller instance of the same problem. **Combine operation:** one comparison between the two minima and one between the two maxima.

Recurrence and expected comparisons. Let T(n) be the number of comparisons for n = 2^k elements:

$$T(2) = 1, \quad T(n) = 2\cdot T(n/2) + 2 \quad \text{for } n > 2$$

Unrolling: the recursion has n/2 leaf pairs, each costing 1, and 2 + 4 + … + n/2 = n − 2 comparisons are spent in the combine steps of the internal nodes. Hence

$$T(n) = n/2 + (n - 2) = \mathbf{3n/2 - 2}$$

For general n the count is $\lceil 3n/2 \rceil - 2$ in the best arrangement. **Comparison with independent search:** $2(n - 1) = 2n - 2$. The ratio $(3n/2 - 2)/(2n - 2) \rightarrow 3/4$, i.e. the tournament method saves about 25 % of the comparisons. Both are $\Theta(n)$; the gain is in the constant factor, because every element is compared once to decide "who is smaller" and the winners/losers are reused, so no element is compared separately for both roles.

Student B – Experimental validation

Datasets: (i) random distinct integers, (ii) ascending, (iii) descending, (iv) all-equal. **Instrumentation:** a counter incremented at every element-to-element comparison. **Sizes:** n = 2, 4, …, 1024 (powers of two) and n = 10, 100, 1000, 5000 (not powers of two).

Table 1.1 – Comparison counts for Max/Min

n	Independent 2(n−1)	Predicted ⌈3n/2⌉−2	Observed D&C (random)	Observed (asc / desc / equal)
2	2	1	1	1 / 1 / 1
4	6	4	4	4 / 4 / 4
8	14	10	10	10 / 10 / 10
16	30	22	22	22 / 22 / 22
32	62	46	46	46 / 46 / 46
64	126	94	94	94 / 94 / 94
128	254	190	190	190 / 190 / 190
256	510	382	382	382 / 382 / 382
512	1,022	766	766	766 / 766 / 766
1,024	2,046	1,534	1,534	1,534 / 1,534 / 1,534
10	18	13	14	14 / 14 / 14
100	198	148	162	162 / 162 / 162
1,000	1,998	1,498	1,510	1,510 / 1,510 / 1,510

n	Independent 2(n−1)	Predicted ⌈3n/2⌉−2	Observed D&C (random)	Observed (asc / desc / equal)
5,000	9,998	7,498	7,950	7,950 / 7,950 / 7,950

Findings. (1) For every n = 2^k the observed count equals 3n/2 − 2 exactly, so the prediction is confirmed. (2) The count is identical for all four datasets: the algorithm performs the same comparisons regardless of data order, so there is no best or worst case. (3) The D&C count is always below 2(n − 1).

Discrepancy. For n not a power of two the observed count is slightly larger than ⌈3n/2⌉ − 2 (n = 10: 14 vs 13; n = 100: 162 vs 148; n = 5000: 7950 vs 7498). The formula assumes every leaf is a pair. With an arbitrary n the midpoint split also creates leaves of size 1 (e.g. 5 → 2 + 3 → 2 + (1 + 2)); a size-1 leaf costs 0 but its parent still spends 2 comparisons to merge it, so the cost is governed by T(n) = T(⌊n/2⌋) + T(⌈n/2⌉) + 2 with T(1) = 0, T(2) = 1, which is larger than the idealised value (about 1.59n for n = 5000). It remains well below 2(n − 1) = 9,998, and splitting so that the leaves are pairs would recover ⌈3n/2⌉ − 2.

Part B – Binary Search

Shared group work – strategy and implementation

Binary search requires the array to be **sorted**. Compare the target with the middle element; the comparison tells us in which half the target can lie, so the other half is discarded and the problem size halves.

```
ALGORITHM BinarySearch(A, low, high, target)
INPUT:
  A – sorted array; low, high – current search boundaries; target – value to locate
OUTPUT:
  position of target, or NOT_FOUND
BEGIN
  IF low > high THEN
    RETURN NOT_FOUND                    // base case: empty search space
  END IF
  mid ← FLOOR((low + high) / 2)
  IF A[mid] = target THEN
```

```
  RETURN mid                        // base case: found
 ELSE IF target < A[mid] THEN
  RETURN BinarySearch(A, low, mid - 1, target)
 ELSE
  RETURN BinarySearch(A, mid + 1, high, target)
 END IF
END
```

Manual trace – A = [3, 8, 12, 17, 21, 26, 34]

Target	low	high	mid	A[mid]	Decision
26	0	6	3	17	26 > 17 → low ← 4
26	4	6	5	26	found → RETURN 5 (2 probes)
10	0	6	3	17	10 < 17 → high ← 2
10	0	2	1	8	10 > 8 → low ← 2
10	2	2	2	12	10 < 12 → high ← 1
10	2	1	—	—	low > high → NOT_FOUND (3 probes)

Student A – Experimental investigation

Test design. Array A = [0, 2, 4, …, 2(n−1)]. Successful searches: every stored value (all n keys for n ≤ 65,536; 20,000 random keys for n = 2^20). Unsuccessful searches: every odd value (lies between elements, so the search must run to an empty range). A "comparison" is one probe of A[mid].

Table 1.2 – Probes in Binary Search

n	Success: best	Success: average	Success: worst	Unsuccessful: average	Unsuccessful: max	$\lfloor \log_2 n \rfloor + 1$
16	1	3.38	5	4.12	5	5
256	1	7.04	9	8.01	9	9
4,096	1	11	13	12	13	13
65,536	1	15	17	16	17	17
1,048,576	7	19.02	20	20	20	21

Evidence. Multiplying n by 16 in each row (16 → 256 → 4096 → 65,536) adds a constant 4 probes each time; the worst case equals $\lfloor \log_2 n \rfloor + 1$ exactly; the best case is 1 probe (target at the middle). At n = 2^20 about one million elements are resolved in at most 20 probes, where linear search needs up to 1,048,576 (the best value 7 and worst value 20 at n = 2^20 come from the 20,000-key sample, which did not contain the true extremes). Unsuccessful searches need about one probe more than successful ones on average.

Student B – Elimination logic

- **Decision rule:** compare target with A[mid]. Equal → stop. target < A[mid] → continue in A[low..mid−1]. target > A[mid] → continue in A[mid+1..high].
- **Portion eliminated:** the half that cannot contain the target, together with A[mid] itself – at least $\lfloor n/2 \rfloor$ elements per probe.
- **Why the target cannot be there:** A is sorted in non-decreasing order, so every element to the right of mid is ≥ A[mid]. If target < A[mid], then target is smaller than all of them and cannot equal any. The symmetric argument holds for the left half. (Invariant: if the target is present in A, it lies in A[low..high].)
- **Termination:** each step either returns or strictly shrinks high − low (new range excludes mid), so the range eventually becomes empty (low > high) and NOT_FOUND is returned.
- **Efficiency:** $T(n) = T(n/2) + 1$, $T(1) = 1$, so $T(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$. Halving k times reduces n to 1 when $k = \log_2 n$; each halving costs only one comparison, so work is logarithmic and the sorted order is exploited fully.

Q2 – The Quality of Decomposition in Sorting (7 Marks)

Shared group work

Implemented: Merge Sort (split by position, merge sorted halves) and Quick Sort. **Quick Sort pivot strategy (clearly stated): the first element of the (sub)array is the pivot.** The partition produces a list of elements smaller than the pivot (Left) and a list of the remaining elements (Right), preserving order; the pivot is placed between the two sorted parts. All values in the experiments are distinct.

```
ALGORITHM MergeSort(A)
INPUT: array A
OUTPUT: sorted array
BEGIN
 IF LENGTH(A) ≤ 1 THEN RETURN A
 mid ← FLOOR(LENGTH(A) / 2)
 L ← MergeSort(A[0..mid-1])
 R ← MergeSort(A[mid..LENGTH(A)-1])
 RETURN Merge(L, R)
END
```

```
ALGORITHM Merge(L, R)
INPUT: sorted arrays L and R
OUTPUT: sorted array Result
BEGIN
 i ← 0 ; j ← 0 ; Result ← empty
 WHILE i < LENGTH(L) AND j < LENGTH(R) DO
 IF L[i] ≤ R[j] THEN
 ADD L[i] to Result ; i ← i + 1
 ELSE
 ADD R[j] to Result ; j ← j + 1
 END IF
 END WHILE
 ADD remaining elements of L and R to Result
 RETURN Result
END
```

```
ALGORITHM QuickSort(A)
INPUT: array A of distinct elements          OUTPUT: sorted array
BEGIN
 IF LENGTH(A) ≤ 1 THEN RETURN A
 pivot ← A[0]                          // pivot strategy: first element
 Left ← empty ; Right ← empty
 FOR i ← 1 TO LENGTH(A) - 1 DO
  IF A[i] < pivot THEN ADD A[i] to Left ELSE ADD A[i] to Right END IF
 END FOR                               // n − 1 comparisons for this partition
 RETURN QuickSort(Left) followed by [pivot] followed by QuickSort(Right)
END
```

Experiment design. For n ∈ {127, 255, 511, 1023, 2047} four inputs are sorted: **balanced** (arranged so that the first element is always the median of its subarray), **random**, **sorted ascending** and **sorted descending**. Counted: comparisons, maximum recursion depth (levels, counting the leaf level) and partition sizes.

Student A – Quick Sort investigation

Input producing balanced partitions. Build the permutation recursively: Balanced(v) = [median of v] + Balanced(lower half) + Balanced(upper half). For v = 0..6 this gives [3, 1, 0, 2, 5, 4, 6]: the first pivot 3 splits into 3 smaller and 3 larger, and the same holds in each half. **Input producing severely unbalanced partitions.** With the first element as pivot, an already sorted (or reverse-sorted) array always has the smallest (largest) element as pivot, so each partition is (0, n−1) or (n−1, 0).

Table 2.1 – First three partition sizes (left, right) for n = 1023

Input	Level-1 split	Level-2 split	Level-3 split	Avg. largest part / n
balanced	(511,511)	(255,255)	(127,127)	0.5
random	(152,870)	(60,91)	(11,48)	0.845
sorted	(0,1022)	(0,1021)	(0,1020)	1
reverse	(1022,0)	(1021,0)	(1020,0)	1

Table 2.2 – Comparisons and maximum recursion depth

n	Input	Quick Sort comparisons	Quick Sort max depth	Merge Sort comparisons	Merge Sort max depth
127	balanced	642	7	584	8
127	random	946	17	725	8
127	sorted	8,001	127	441	8
127	reverse	8,001	127	447	8
255	balanced	1,538	8	1,364	9
255	random	2,488	20	1,731	9

n	Input	Quick Sort comparisons	Quick Sort max depth	Merge Sort comparisons	Merge Sort max depth
255	sorted	32,385	255	1,016	9
255	reverse	32,385	255	1,023	9
511	balanced	3,586	9	3,113	10
511	random	4,877	20	3,962	10
511	sorted	130,305	511	2,295	10
511	reverse	130,305	511	2,303	10
1,023	balanced	8,194	10	6,990	11
1,023	random	11,043	23	8,970	11
1,023	sorted	522,753	1023	5,110	11
1,023	reverse	522,753	1023	5,119	11
2,047	balanced	18,434	11	15,513	12
2,047	random	23,367	22	19,911	12
2,047	sorted	2,094,081	2047	11,253	12
2,047	reverse	2,094,081	2047	11,263	12

Observations. Balanced input: depth = $\log_2(n+1)$ (10 for n = 1023) and comparisons ≈ $n \cdot \log_2 n$. Sorted/reverse input: depth = n (1023) and comparisons = $n(n-1)/2$ = 522,753 for n = 1023 – exactly the quadratic value, about 64× the balanced count. Random input falls in between (depth 23, comparisons ≈ 1.35× the balanced count) because random pivots give splits of unequal but not extreme sizes.

How pivot selection influences decomposition. The pivot decides the sizes of the two subproblems: the recursion tree is balanced only if the pivot is close to the median of the current subarray. A pivot that is the minimum or maximum removes only one element per level. Choosing the first element makes the decomposition depend on the input order (so presorted data – very common in practice – is the worst case). Strategies that reduce this dependence are a random pivot, median-of-three, or the exact median (e.g. Median of Medians, Q6).

Student B – Comparative divide-and-conquer analysis

- **Merge Sort decomposition:** split by position into ⌊n/2⌋ and ⌈n/2⌉, independent of the values. Work is moved into the combine (Merge) step: $T(n) = 2T(n/2) + n \rightarrow \Theta(n \log n)$ in every case. Observed depth is always $\lceil \log_2 n \rceil + 1$ (8, 9, 10, 11, 12 for the five sizes) for every input.
- **Quick Sort decomposition:** split by value around the pivot; the work (partition) is done before the recursive calls and the combine step is trivial concatenation. $T(n) = T(k) + T(n-k-1) + (n-1)$, where k is the size of the left part, so the recursion shape depends on the data.
- **Why Merge Sort is more predictable:** its split sizes are fixed by n alone, so the recursion tree always has height $\lceil \log_2 n \rceil$ and each level costs ≤ n. Quick Sort's split sizes depend on pivot values; the same n can give depth 10 (balanced), 23 (random) or 1023 (sorted).
- **Partition quality and tree height:** if each partition keeps at least a fraction α of the elements on the smaller side, the height is at most log base $1/(1-\alpha)$ of n. For a 50:50 split this is $\log_2 n$; for a 90:10 split it is $\log_{10/9} n \approx 6.6 \cdot \log_2 n$ – still O(log n), so Quick Sort stays O(n log n) with a larger constant. Only when the smaller part has O(1) elements does the height become linear.
- **How repeated imbalance gives $O(n^2)$:** if every partition splits into sizes 0 and n−1, $T(n) = T(n-1) + (n-1) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = O(n^2)$, the recursion tree is a chain of height n, and the stack depth is also n (observed: 2,094,081 comparisons and depth 2,047 for n = 2047).

Joint design question

If the characteristics of the input are not known in advance, what information would we want before choosing between Merge Sort and Quick Sort?

- **Input size n** – for small n (≈ 10–30) either is fine; insertion sort may be better still.
- **Degree of pre-sortedness / pattern** – sorted, reversed, nearly sorted or organ-pipe data ruin a first-element Quick Sort; a random/median-of-three pivot or Merge Sort is needed.
- **Duplicates and key distribution** – many equal keys hurt simple two-way partitioning (need 3-way partition).
- **Is a worst-case guarantee or adversarial input a concern?** If response time must be bounded (real-time, public-facing service) choose Merge Sort (or Introsort).
- **Memory available** – Merge Sort needs O(n) extra space; in-place Quick Sort needs O(log n) stack when the smaller side is recursed first.
- **Stability required?** Merge Sort is stable; Quick Sort is not.

- **Data representation** – arrays (cache-friendly, favours Quick Sort) versus linked lists or external/disk data (favours Merge Sort); cost of a comparison; whether parallelism is available.

Without any of this information, the safest default is a method with a guaranteed O(n log n) bound (Merge Sort, or Introsort – Quick Sort that falls back to Heap Sort when depth exceeds $2 \cdot \log_2 n$).

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication (7 Marks)

Shared group work

Two n-digit integers are represented in decimal. **Standard multiplication** multiplies every digit of x by every digit of y: n^2 single-digit multiplications. **Karatsuba** splits each number into a high and a low half and uses three recursive multiplications instead of four.

```
ALGORITHM Karatsuba(x, y)
INPUT:  non-negative integers x and y
OUTPUT: the product x × y
BEGIN
 IF x < 10 OR y < 10 THEN
   RETURN x × y                          // base case: a single-digit operand
 END IF
 n ← maximum(Digits(x), Digits(y))
 m ← FLOOR(n / 2)
 a ← FLOOR(x / 10^m) ;  b ← x MOD 10^m      // x = a·10^m + b
 c ← FLOOR(y / 10^m) ;  d ← y MOD 10^m      // y = c·10^m + d
 z2 ← Karatsuba(a, c)
 z0 ← Karatsuba(b, d)
 z1 ← Karatsuba(a + b, c + d) - z2 - z0      // equals a·d + b·c
 RETURN z2 × 10^(2m) + z1 × 10^m + z0        // combine: shifts and additions only
END
```

Standard multiplication (schoolbook): for every digit x_i and every digit y_j add $x_i \cdot y_j$ to position i + j of a result array, then propagate carries – n^2 digit products. Both algorithms were implemented (Appendix B) and verified against the built-in product for every test pair.

Student A – Algorithmic derivation

Decomposition. Let x and y have n digits and $m = \lfloor n/2 \rfloor$. Write $x = a \cdot 10^m + b$ and $y = c \cdot 10^m + d$, where a, c are the high halves and b, d the low halves. Then

$$x \cdot y = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd$$

Subproblems. The direct expression needs four half-size products (ac, ad, bc, bd): $T(n) = 4T(n/2) + O(n)$, which by the Master Theorem ($n^{\log_2 4} = n^2$) is $\Theta(n^2)$ – no better than the standard method.

Karatsuba's observation. (a + b)(c + d) = ac + ad + bc + bd, so ad + bc = (a + b)(c + d) − ac − bd. Only **three** half-size products are needed: z2 = ac, z0 = bd and z3 = (a+b)(c+d). The extra work – two additions before, two subtractions and shifts after – is O(n).

Recurrence. $T(n) = 3 \cdot T(n/2) + cn, \quad T(1) = 1$

Solution. Master Theorem with a = 3, b = 2, f(n) = cn: $n^{(\log_2 3)} \approx n^{1.585}$ grows faster than n, so Case 1 applies and $\mathbf{T(n) = \Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585})}$. Interpretation by levels: level i has 3^i subproblems of size n/2^i, costing $3^i \cdot c \cdot n/2^i = cn \cdot (3/2)^i$; the geometric series is dominated by the leaves, $3^{(\log_2 n)} = n^{(\log_2 3)}$.

Worked example, 1234 × 5678 (n = 4, m = 2). a = 12, b = 34, c = 56, d = 78. z2 = 12·56 = 672; z0 = 34·78 = 2,652; (a+b)(c+d) = 46·134 = 6,164; z1 = 6,164 − 672 − 2,652 = 2,840. Result = $672 \cdot 10^4 + 2{,}840 \cdot 10^2 + 2{,}652 = 6{,}720{,}000 + 284{,}000 + 2{,}652 =$ **7,006,652** = 1234 × 5678 ✓.

Student B – Crossover investigation

Experiment design. Random n-digit operands for n = 4, 8, 16, …, 1024 digits (one pair per size, same pair given to both algorithms). Each timing is repeated (5 repetitions for n ≤ 256, 3 for larger) and the **minimum** is reported to reduce operating-system noise. Multiplications = single-digit products. Environment: Python 3, pure-Python digit arithmetic for the standard method.

Table 3.1 – Multiplication counts and running times

Digits n	Standard mults (n^2)	Karatsuba mults	Mult. ratio	Standard time	Karatsuba time	Faster
4	16	15	1.07×	6 µs	10 µs	Standard
8	64	53	1.21×	14 µs	33 µs	Standard
16	256	140	1.83×	37 µs	88 µs	Standard
32	1,024	390	2.63×	132 µs	235 µs	Standard
64	4,096	1,147	3.57×	491 µs	709 µs	Standard
128	16,384	3,637	4.50×	2.04 ms	2.36 ms	Standard
256	65,536	10,887	6.02×	8.83 ms	6.94 ms	Karatsuba
512	262,144	33,073	7.93×	38.78 ms	21.53 ms	Karatsuba
1,024	1,048,576	98,804	10.61×	169.33 ms	65.87 ms	Karatsuba

Crossover. In terms of **multiplication count** Karatsuba is already ahead at n = 4 (15 vs 16) and the gap widens to about 10.6× at n = 1024. In terms of **running time** Standard is faster up to n = 128 digits (2.04 ms vs 2.36 ms) and Karatsuba is faster from n = 256 digits (6.9 ms vs 8.8 ms): the approximate crossover range is **128–256 digits** (for this Python implementation; it would differ with other languages or a tuned base case).

Why Karatsuba can be worse for small inputs, and the overhead. (1) Every recursive call splits numbers (divisions/modulus by 10^m), counts digits, and performs two additions, two subtractions and shifted additions; these O(n) operations have large constants relative to a handful of digit products. (2) The recursion creates 3^k calls with function-call and temporary-number overhead; near the leaves (single digits) the work per call is tiny compared with the call overhead. (3) a + b and c + d can have one more digit than the halves, so subproblems are slightly larger than n/2 (the observed count 98,804 for n = 1024 exceeds the ideal 3^10 = 59,049). A practical implementation therefore switches to the standard method below a cut-off size.

Joint conclusion

Why does better asymptotic complexity not automatically imply better performance for every input size? Big-O hides constant factors and lower-order terms and describes behaviour only as $n \to \infty$. Karatsuba's true cost is roughly $c_1 \cdot n^{1.585}$ with a large c_1 (additions, subtractions, splitting, recursion overhead), while standard multiplication is $c_2 \cdot n^2$ with a very small c_2. The curves cross only at some finite n^* where $n^{*0.415} \approx c_1/c_2$; below n^* the "worse" algorithm wins. Our measurements show this directly: fewer multiplications from n = 4, but a lower running time only from about 128–256 digits.

Q4 – The Reality of Strassen's Matrix Multiplication (7 Marks)

Shared group work

For n × n matrices (n a power of two) the product C = A·B is computed by partitioning each matrix into four n/2 × n/2 blocks. Both the conventional block algorithm and Strassen's algorithm were implemented (Appendix B) and checked against each other for random integer matrices with n = 2 … 128 (identical results in every case) and for non-power-of-two sizes (n = 50, 100, 129) using padding.

```
ALGORITHM Strassen(A, B, n)
INPUT:  n × n matrices A and B, n a power of 2
OUTPUT: C = A × B
BEGIN
 IF n = 1 THEN
   RETURN [A[0][0] × B[0][0]]              // base case
 END IF
 h ← n / 2 ; partition A into A11, A12, A21, A22 and B into B11, B12, B21, B22
 M1 ← Strassen(A11 + A22, B11 + B22, h)
 M2 ← Strassen(A21 + A22, B11, h)
 M3 ← Strassen(A11, B12 - B22, h)
 M4 ← Strassen(A22, B21 - B11, h)
 M5 ← Strassen(A11 + A12, B22, h)
 M6 ← Strassen(A21 - A11, B11 + B12, h)
 M7 ← Strassen(A12 - A22, B21 + B22, h)
 C11 ← M1 + M4 - M5 + M7
 C12 ← M3 + M5
 C21 ← M2 + M4
 C22 ← M1 - M2 + M3 + M6
 RETURN matrix formed by blocks C11, C12, C21, C22
END

ALGORITHM StrassenAnySize(A, B, n)          // padding wrapper
 s ← smallest power of 2 with s ≥ n
 A′ ← A padded with zero rows/columns to s × s ;  B′ ← B padded likewise
 C′ ← Strassen(A′, B′, s)
 RETURN top-left n × n block of C′
```

Student B – Algebraic design and correctness

Conventional block decomposition. C11 = A11B11 + A12B21, C12 = A11B12 + A12B22, C21 = A21B11 + A22B21, C22 = A21B12 + A22B22.

The eight standard block multiplications: A11B11, A12B21, A11B12, A12B22, A21B11, A22B21, A21B12, A22B22. This gives $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.

The transformation to seven. Strassen forms seven products from sums/differences of blocks (M1…M7 above) and recovers the four result blocks using only additions and subtractions. The idea is the same as Karatsuba: spend a few cheap $O(n^2)$ additions to save one expensive recursive multiplication.

Why the expressions are correct (substitute and cancel):

- C12 = M3 + M5 = A11(B12 − B22) + (A11 + A12)B22 = A11B12 − A11B22 + A11B22 + A12B22 = **A11B12 + A12B22 ✓**
- C21 = M2 + M4 = (A21 + A22)B11 + A22(B21 − B11) = A21B11 + A22B11 + A22B21 − A22B11 = **A21B11 + A22B21 ✓**
- C11 = M1 + M4 − M5 + M7 = (A11B11 + A11B22 + A22B11 + A22B22) + (A22B21 − A22B11) − (A11B22 + A12B22) + (A12B21 + A12B22 − A22B21 − A22B22) = **A11B11 + A12B21 ✓** (all other terms cancel in pairs).
- C22 = M1 − M2 + M3 + M6 = (A11B11 + A11B22 + A22B11 + A22B22) − (A21B11 + A22B11) + (A11B12 − A11B22) + (A21B11 + A21B12 − A11B11 − A11B12) = **A21B12 + A22B22 ✓**

Block multiplication is not commutative, so the order of factors in each product is preserved exactly as written. The correctness was also confirmed numerically (identical to the conventional result for all test matrices).

Recurrence. Seven recursive multiplications plus 18 matrix additions/subtractions of size n/2: $T(n) = 7T(n/2) + 18(n/2)^2 = 7T(n/2) + \Theta(n^2) \rightarrow$ **$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$**.

Padding. Strassen's recursion needs even sizes at every level. If n is not a power of two, A and B are padded with zero rows and columns up to the next power of two s; the product of the padded matrices has the true product in its top-left n × n block, the rest is zeros and is discarded. Cost: the work is that of an s × s problem, up to 8× more than for n × n when n is just above a power of two.

Student A – Experimental and engineering analysis

Experiment design. Random integer matrices (entries −9…9), n = 2, 4, …, 128; scalar multiplications and additions/subtractions counted by instrumentation; running times are the best of 3 runs (1 run for n = 128). Three Strassen variants were timed: recursion to 1 × 1 ("pure"), and hybrids that switch to the conventional algorithm at block size ≤ 8, 16 or 32. Implementation: pure Python lists (no BLAS).

Table 4.1 – Operation counts

n	Conventional mults (n^3)	Strassen mults (7^k)	Mult. ratio	Conventional adds	Strassen adds/subs
2	8	7	0.875	4	18
4	64	49	0.766	48	198
8	512	343	0.670	448	1,674
16	4,096	2,401	0.586	3,840	12,870
32	32,768	16,807	0.513	31,744	94,698
64	262,144	117,649	0.449	258,048	681,318
128	2,097,152	823,543	0.393	2,080,768	4,842,954

Table 4.2 – Running times (pure Python)

n	Conventional	Strassen (pure)	Hybrid cut-off 8	Hybrid cut-off 16	Hybrid cut-off 32
2	3 µs	21 µs	–	–	–
4	13 µs	160 µs	–	–	–
8	66 µs	1.22 ms	–	–	–
16	417 µs	9.45 ms	666 µs	–	–
32	2.89 ms	104.87 ms	9.35 ms	6.37 ms	–
64	21.22 ms	470.06 ms	40.08 ms	27.95 ms	23.35 ms
128	163.49 ms	3.85 s	322.01 ms	230.73 ms	183.52 ms

Table 4.3 – Non-power-of-two sizes (padding), Strassen with cut-off 16

n	Padded size	Extra work vs. true n^3 (%)	Conventional time	Strassen time
50	64	52.3 %	10.56 ms	28.51 ms
100	128	52.3 %	83.25 ms	221.56 ms
129	256	87.2 %	253.70 ms	1.69 s

Practical crossover range. (a) **Multiplication count:** Strassen needs fewer multiplications at every size (ratio (7/8)^k → 0.39 at n = 128). (b) **Total arithmetic operations** (multiplications + additions, computed from $T(n) = 7T(n/2) + 18(n/2)^2$): pure Strassen overtakes the conventional method only at n ≈ **1024**; with a cut-off of 32 it overtakes at n ≈ **64**. (c) **Measured running time:** within the sizes tested (n ≤ 128) pure Strassen is never faster (23× slower at n = 128: 3.85 s vs 0.16 s); the hybrid with cut-off 32 comes within about 12 % of the conventional time at n = 128 (0.184 s vs 0.163 s) and is the only variant approaching the crossover, so in this implementation the crossover lies **above n = 128 (expected in the few-hundreds range with a tuned cut-off)** – larger sizes were not run because pure-Python Strassen becomes very slow.

Overheads. (1) **Additions/subtractions:** Strassen performs 18 block additions per level; at n = 128 it performs 4.84 million additions against 2.08 million for the conventional method, more than offsetting the saved multiplications (0.82 million vs 2.10 million). (2) **Recursion:** 7^k leaf calls (823,543 at n = 128) each with function-call and list-slicing cost; the cut-off removes most of this (pure 3.85 s → 0.18 s). (3) **Padding:** an n = 129 matrix is padded to 256 (87 % of the work is on padding), and measured Strassen time is 6.7× the conventional time. (4) **Memory/cache:** each level allocates 10 temporary operand matrices and 7 product matrices, giving extra memory of the order of n^2 and poorer locality than the in-place conventional loops. In pure Python the effect of the cache could not be separated from interpreter overhead, so we report it as a qualitative expectation rather than a measured result.

Joint engineering decision

When would we use conventional multiplication although Strassen has better asymptotic complexity?

- **Small or moderate matrices** (below the measured/tuned crossover, typically up to a few hundred): overhead dominates; the conventional method is faster.
- **Size not close to a power of two:** padding wastes up to 87 % of the work (n = 129).

- **Optimised libraries/hardware:** BLAS, SIMD, GPUs and cache-blocked kernels make the $O(n^3)$ method extremely fast per operation; their constant factors beat Strassen until very large n.
- **Memory limited:** Strassen needs $O(n^2)$ extra temporaries.
- **Floating-point accuracy matters:** Strassen has weaker numerical stability (errors are not bounded entry-wise, only norm-wise).
- **Sparse, banded or structured matrices:** specialised algorithms exploit zeros; Strassen destroys the structure.
- **Simplicity / maintainability:** the conventional algorithm is simple, parallelises trivially and is easy to verify.

Strassen (usually as a hybrid with a cut-off) is worthwhile for very large, dense, roughly power-of-two matrices when memory is available and exact arithmetic or modest accuracy loss is acceptable.

Q5 – Upgrading Closest Pair Using Divide and Conquer (6 Marks)

Shared group work

Review of the brute-force approach (CO2). Compute the distance for every pair of points (i, j), i < j, and keep the smallest: $n(n-1)/2$ distance computations, $\Theta(n^2)$.

```
ALGORITHM ClosestPair(P)
INPUT:  set P of n ≥ 2 points with distinct coordinates
OUTPUT: smallest distance between any two points of P
BEGIN
 Px ← points of P sorted by x-coordinate          // sorted once
 Py ← points of P sorted by y-coordinate          // sorted once
 RETURN ClosestRec(Px, Py)
END

ALGORITHM ClosestRec(Px, Py)
INPUT:  the same point set, held sorted by x (Px) and sorted by y (Py)
OUTPUT: smallest distance in the set
BEGIN
 n ← LENGTH(Px)
 IF n ≤ 3 THEN
  RETURN smallest Distance over all pairs of Px      // base case: brute force
 END IF
 mid ← FLOOR(n / 2) ;  midX ← Px[mid].x
 Pl ← Px[0..mid-1] ;  Pr ← Px[mid..n-1]
 Yl ← points of Py that belong to Pl (order kept) ;  Yr ← the other points of Py
 dl ← ClosestRec(Pl, Yl)
 dr ← ClosestRec(Pr, Yr)
 d ← MIN(dl, dr)
 Strip ← points of Py with |p.x - midX| < d              // already sorted by y
 FOR i ← 0 TO LENGTH(Strip) - 1 DO
  j ← i + 1
  WHILE j < LENGTH(Strip) AND Strip[j].y - Strip[i].y < d DO
   d ← MIN(d, Distance(Strip[i], Strip[j]))
   j ← j + 1
  END WHILE
```

```
  END FOR
  RETURN d
END
```

Student A – Decomposition design

- **Organisation of the points:** sorted by x (to divide by a vertical line) and sorted by y (to scan the strip quickly). Sorting is done once, O(n log n); the y-order is passed down by filtering, which keeps each combine step O(n).
- **Division:** the vertical line x = midX separates the left half (first ⌊n/2⌋ points) from the right half.
- **Each recursive call returns** the smallest distance inside its half (and receives its points in both sorted orders).
- **Combine:** d = min(dl, dr) is not final, because the closest pair may have one point in each half. Only points within distance d of the dividing line can form a pair closer than d, so these are collected in a strip, ordered by y, and checked.
- **Why the boundary needs special treatment:** the recursion never compares a left point with a right point. A close pair straddling the line is invisible to both recursive calls, so it must be searched for explicitly in the combine step.

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$ (plus the initial O(n log n) sort); brute force is $\Theta(n^2)$.

Student B – Comparative evidence

Equivalent datasets: the same set of n distinct random points in $[0, 10^6)^2$ is given to both algorithms for n = 10, 20, 50, 100, 200, 500, 1000, 2000, 4000; the two results were asserted equal in every run. Distance computations were counted by instrumenting the distance function; running time is the best of 3 runs (1 run for n ≥ 2000) and includes the sorting in D&C.

Table 5.1 – Brute force vs. divide and conquer

n	Brute-force distance calcs	D&C distance calcs	Reduction	Brute-force time	D&C time	Faster
10	45	10	4.5×	20 µs	29 µs	Brute
20	190	26	7.3×	73 µs	68 µs	D&C

n	Brute-force distance calcs	D&C distance calcs	Reduction	Brute-force time	D&C time	Faster
50	1,225	70	17.5×	474 µs	188 µs	D&C
100	4,950	121	40.9×	1.82 ms	418 µs	D&C
200	19,900	267	74.5×	8.99 ms	1.09 ms	D&C
500	124,750	560	222.8×	43.96 ms	2.26 ms	D&C
1,000	499,500	1,124	444.4×	234.60 ms	8.04 ms	D&C
2,000	1,999,000	2,297	870.3×	703.23 ms	10.25 ms	D&C
4,000	7,998,000	4,633	1726.3×	2.83 s	24.80 ms	D&C

Growth. Brute-force work grows by a factor of 4 whenever n doubles (n = 1000 → 2000: 499,500 → 1,999,000; 4000: 7,998,000), the signature of $\Theta(n^2)$. D&C distance calculations grow almost linearly (1,124 → 2,297 → 4,633, about $1.15 \cdot n$), because only a handful of strip points are examined per point; the dominant remaining cost is sorting/splitting, giving n log n growth in time (8.0 ms → 10.3 ms → 24.8 ms).

Practical advantage. D&C needs fewer distance computations from n = 10 (10 vs 45). In running time brute force is still as fast at n = 20 (73 µs vs 68 µs, essentially equal) and D&C is clearly faster from n = 50 (0.19 ms vs 0.47 ms) – so D&C becomes practically advantageous at **about 20–50 points**, and is about 114× faster at n = 4000 (25 ms vs 2.8 s).

Cross-understanding

Student B – why does the strip stage not need to compare each point with every other point? Within the strip the points are ordered by y, and d is the smallest distance found so far. A point can only be part of a closer pair with a point whose y-difference is less than d, so the inner loop stops as soon as the y-gap reaches d. Any such candidate lies in a d × 2d rectangle (d on each side of the dividing line, height d above the point). That rectangle can be cut into eight (d/2 × d/2) squares, each containing at most one point (the diagonal of a square is $d/\sqrt{2} < d$, so two points in one square would be closer than d, yet points in the same half are at least d apart). So at most 7 other points need to be checked per point – a constant – and the strip costs $O(n)$, not $O(n^2)$.

Student A – what evidence shows that D&C reduced the work? Table 5.1: for identical datasets the number of distance computations drops from $n(n-1)/2$ to about $1.15 \cdot n$ (e.g. 7,998,000 → 4,633

at n = 4000, a 1,726× reduction), the doubling of n quadruples brute-force work but only about doubles D&C work, and the measured running time falls from 2.8 s to 25 ms for n = 4000 with an identical answer.

Q6 – Advanced Divide-and-Conquer Mini-Investigations (7 Marks)

Student A – Median of Medians (deterministic selection)

Task: find the k-th smallest element of an unsorted array without sorting it (O(n log n)). **QuickSelect** partitions around a pivot and continues only in the part containing rank k; its cost depends entirely on pivot quality.

```
ALGORITHM Select(A, k)                    // Median of Medians
INPUT:  array A of distinct elements; k – rank wanted (0 = smallest)
OUTPUT: the k-th smallest element of A
BEGIN
 n ← LENGTH(A)
 IF n ≤ 5 THEN
   SORT A ;  RETURN A[k]                  // base case
 END IF
 Medians ← empty list
 FOR each group G of 5 consecutive elements of A (last group may be smaller) DO
   SORT G ;  ADD G[FLOOR(LENGTH(G) / 2)] to Medians
 END FOR
 pivot ← Select(Medians, FLOOR((LENGTH(Medians) - 1) / 2))   // median of the medians
 Less ← elements of A smaller than pivot ;  Greater ← elements of A larger than pivot
 IF k < LENGTH(Less) THEN
   RETURN Select(Less, k)
 ELSE IF k = LENGTH(Less) THEN
   RETURN pivot
 ELSE
   RETURN Select(Greater, k - LENGTH(Less) - 1)
 END IF
END
```

- **Grouping strategy:** the n elements are split into ⌈n/5⌉ groups of five; each group is sorted (constant time) and its median extracted.
- **How the pivot is determined:** the pivot is the median of these ⌈n/5⌉ medians, found by a recursive call of the same algorithm on an array of size n/5.

- **Why disastrous partitions are prevented:** at least half of the group medians (≈ n/10 of them) are ≤ pivot, and each of those medians has two smaller elements in its group, so at least 3·(n/10) = **3n/10 elements are ≤ pivot**; symmetrically at least 3n/10 are ≥ pivot. Hence each side of the partition has **at most 7n/10** elements, whatever the data order.
- **Worst-case guarantee:** $T(n) \leq T(n/5) + T(7n/10) + cn$. Because $1/5 + 7/10 = 9/10 < 1$, assume $T(m) \leq am$ for $m < n$: $T(n) \leq an/5 + 7an/10 + cn = (9a/10 + c)n \leq an$ whenever $a \geq 10c$. Therefore **T(n) = O(n) in the worst case**.

Table 6.1 – Selection experiments (comparisons; distinct integers; sorted input, k = largest; random input, k = median)

n	QuickSelect, first pivot, sorted input	Largest part / n (QuickSelect)	Median of Medians, sorted input	Largest part / n (MoM)	MoM, random input	QuickSelect random pivot: mean / max of 30 runs
1,000	499,500	0.999	5,345	0.667	7,247	3,042 / 5,204
2,000	1,999,000	1	11,004	0.533	15,223	7,146 / 11,255
4,000	7,998,000	1	22,472	0.625	31,664	14,410 / 25,376
8,000	31,996,000	1	45,619	0.548	64,185	27,338 / 41,723

Comparison with QuickSelect / deterministic poor-pivot effect. With the first element as pivot and sorted input, each partition removes only one element: comparisons = n(n−1)/2 (499,500 for n = 1000; 31,996,000 for n = 8000 – quadratic, and the largest part is ≈ 100 % of n). On the same input Median of Medians uses 5,345 and 45,619 comparisons (≈ 5.3n – 5.7n, linear) and its largest part never exceeded 0.67n, consistent with the 7n/10 bound.

Why randomisation reduces but does not eliminate poor partitions. A random pivot makes a bad split unlikely for every input (expected O(n) comparisons – here ≈ 3.4n at n = 8000 on average) and removes the dependence on input order. But it is still a gamble: individual runs can be poor (the worst of 30 runs at n = 8000 used 41,723 comparisons, 1.5× the mean, and in principle consecutive unlucky pivots can degrade it towards $O(n^2)$ with tiny probability). Only a deterministic guarantee like Median of Medians bounds the worst case. The price is a larger constant: on random data MoM used 64,185 comparisons versus a mean of 27,338 for random-pivot QuickSelect (n = 8000), so in practice randomised QuickSelect is usually preferred unless worst-case guarantees are essential.

Student B verification – why is pivot quality central to selection? After each partition the algorithm keeps only the side containing rank k, so the total work is the sum of the sizes of the successive subarrays. A pivot near the median shrinks the problem geometrically (n + n/2 + n/4 + … = O(n)); a pivot near an end shrinks it by one element (n + (n−1) + … = O(n²)). The pivot therefore alone decides whether the "discard" step is powerful or useless.

Student B – Meet-in-the-Middle (Subset Sum)

Problem: given n integers and a target T, does some subset sum to T? Brute force tries all 2^n subsets.

```
ALGORITHM MeetInTheMiddle(items, n, target)
INPUT:  array items of n integers; target – required sum
OUTPUT: TRUE if some subset of items sums to target, otherwise FALSE
BEGIN
 h ← FLOOR(n / 2)
 Left ← items[0..h-1] ;  Right ← items[h..n-1]          // divide into two halves
 SumsL ← all 2^h subset sums of Left ;  SumsR ← all 2^(n-h) subset sums of Right
 SORT SumsL in ascending order ;  SORT SumsR in ascending order
 i ← 0 ;  j ← LENGTH(SumsR) - 1
 WHILE i < LENGTH(SumsL) AND j ≥ 0 DO
   s ← SumsL[i] + SumsR[j]
   IF s = target THEN RETURN TRUE
   ELSE IF s < target THEN i ← i + 1                  // need a larger sum
   ELSE j ← j - 1                                     // need a smaller sum
   END IF
 END WHILE
 RETURN FALSE
END
```

- **How the problem is divided:** the item set is split into two halves of n/2 items; every subset of the full set is a union of one subset of each half, so sum = (left sum) + (right sum).
- **Subset results for both halves:** each half has 2^(n/2) subsets; their sums are generated by doubling (for each item, append "old sum + item" to the list).

- **Combining/searching:** a solution exists iff some left sum + some right sum = T. After sorting both lists, a two-pointer scan (smaller left sums with larger right sums) finds such a pair in at most 2·2^(n/2) steps.
- **Search-space size:** 2·2^(n/2) = 2^(n/2 + 1) sums are generated and examined instead of 2^n subsets. Time O(n·2^(n/2)) including sorting.
- **Memory:** both sum lists are stored: 2·2^(n/2) integers = O(2^(n/2)) (e.g. 2,097,152 integers for n = 40) versus O(n) for straightforward enumeration.

Table 6.2 – Meet-in-the-Middle vs. enumeration (unsuccessful search = worst case; random items)

n	Brute force: subsets 2^n	MITM: sums generated	MITM: search steps (max)	MITM memory (integers stored)	Brute-force time	MITM time
10	1,024	64	32	64	902 µs	33 µs
16	65,536	512	256	512	119.85 ms	121 µs
20	1,048,576	2,048	1,024	2,048	1.96 s	720 µs
24	16,777,216	8,192	4,096	8,192	not run (long)	5.46 ms
30	1,073,741,824	65,536	32,768	65,536	not run (infeasible)	43.79 ms
36	68,719,476,736	524,288	262,144	524,288	not run (infeasible)	339.79 ms
40	1,099,511,627,776	2,097,152	1,048,576	2,097,152	not run (infeasible)	1.51 s

Time-memory trade-off. MITM turns the time bound from 2^n to about 2^(n/2) – the "square root" of the work – by paying O(2^(n/2)) memory. At n = 20 it is about 2,700× faster than enumeration (0.72 ms vs 1.96 s); at n = 40 brute force would need 1.1×10^{12} subsets (infeasible), while MITM finishes in 1.5 s using about two million stored sums. Brute force needs almost no memory; MITM is limited by RAM at roughly n ≈ 40–50. Answers of the two methods were identical for all n ≤ 20 (including a subset-sum-exists case).

Student A verification – why can MITM reduce computational work even though it introduces storage? The storage is the stored result of work that would otherwise be repeated: each

half-sum is computed once and then reused in combination with every sum of the other half, so $2^{(n/2)} \times 2^{(n/2)} = 2^n$ combinations are represented by only $2 \cdot 2^{(n/2)}$ stored values, and the sorted order lets the search check them without enumerating them one by one. Work is traded against memory: exponentially less time for exponentially less (but still exponential) space.

Q7 – Design Divide and Conquer from First Principles (10 Marks)

Problem chosen for the Day-4 challenge – Maximum Contiguous Subarray Sum. Given an array A of n integers (positive, zero or negative), find the largest sum of a non-empty contiguous block A[i..j]. (Example: A = [−2, 5, 6, −1, 7, −9, 3] → 5 + 6 − 1 + 7 = 17.)

Note for the pair: Stage 1 below is a worked example of two independent initial proposals. In the submission, Student A and Student B must record the ideas they genuinely wrote down independently before comparing (the assignment awards no penalty for incorrect first attempts); edit Stages 1–2 accordingly.

Stage 1 – Independent design proposals (retained)

Element	**Student A – initial proposal**	**Student B – initial proposal**
Decomposition	Split A into a left half and a right half at the middle.	Split A into two halves at the middle.
Base case	One element: the answer is that element.	One element: return it.
Information from subproblems	Only the best sum inside each half.	The best sum inside the half and the total sum of the half.
Combine strategy	The best subarray is either in the left, in the right, or **crosses the middle**; find the crossing one by scanning outward from the middle in both directions, remembering the best sum on each side; answer = max of the three.	Answer = max(left best, right best, left total + right total).

Stage 2 – Comparison of the two designs

Aspect	**Evaluation**
Where they agree	Both split at the middle, both use a single element as base case, both take the maximum of several candidates.
Where they differ	A recognises the crossing case and finds it by an explicit scan; B tries to obtain the crossing sum cheaply from stored totals.

Aspect	Evaluation
Strength of A	Complete and correct: the three cases (left / right / crossing) cover every subarray. Weakness: the combine step is O(n), so $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$; the scan repeats work the children already did.
Strength of B	Has the right instinct – a child should return more than one number, which makes the combine O(1). Weakness: "left total + right total" is the sum of the **whole** array; the best crossing subarray usually uses only the best **suffix** of the left half and best **prefix** of the right half (e.g. [5, −9 | −8, 6, 7]) – so B's combine is incorrect and misses cases (also misses an all-negative array's correct answer).
Missing information	A: nothing missing for correctness but redundant work. B: the best prefix and the best suffix of each half.
Ideas retained	From **A**: the three-case structure (left, right, crossing) as the basis of correctness. From **B**: return a summary tuple from each subproblem so that the combine is constant-time; extend B's "total" with best prefix and best suffix.

Stage 3 – Joint final design

- **Decomposition:** split A[low..high] at mid = ⌊(low+high)/2⌋.
- **Subproblems:** the left half A[low..mid] and the right half A[mid+1..high] – independent, same problem on half the size.
- **Base case:** one element x: total = prefix = suffix = best = x.
- **Information returned (4-tuple):** total (sum of the whole block), prefix (largest sum of a block starting at its left end), suffix (largest sum of a block ending at its right end), best (largest sum of any block inside).
- **Combine:** total = L.total + R.total; prefix = max(L.prefix, L.total + R.prefix); suffix = max(R.suffix, R.total + L.suffix); best = max(L.best, R.best, **L.suffix + R.prefix**).

```
ALGORITHM MaxSubarray(A, low, high)
INPUT:  array A of integers; low ≤ high – boundaries of the current subarray
OUTPUT: (total, prefix, suffix, best) for A[low..high]
BEGIN
  IF low = high THEN                          // base case
```

```
  RETURN (A[low], A[low], A[low], A[low])
 END IF
 mid ← FLOOR((low + high) / 2)
 L ← MaxSubarray(A, low, mid)
 R ← MaxSubarray(A, mid + 1, high)
 total  ← L.total + R.total
 prefix ← MAX(L.prefix, L.total + R.prefix)          // prefix stays in L or extends into R
 suffix ← MAX(R.suffix, R.total + L.suffix)          // suffix stays in R or extends into L
 best   ← MAX(L.best, R.best, L.suffix + R.prefix)   // inside L, inside R, or crossing mid
 RETURN (total, prefix, suffix, best)
END
// Answer to the problem: MaxSubarray(A, 0, n - 1).best
```

Manual trace for A = [2, −1, 3, −4] (tuple = total, prefix, suffix, best)

Step	Call	Computation	Returns
1	(0,0) and (1,1)	base cases: (2,2,2,2) and (−1,−1,−1,−1)	—
2	combine (0,1)	total = 1; prefix = max(2, 2−1) = 2; suffix = max(−1, −1+2) = 1; best = max(2, −1, 2+(−1)) = 2	(1, 2, 1, 2)
3	(2,2) and (3,3)	base cases: (3,3,3,3) and (−4,−4,−4,−4)	—
4	combine (2,3)	total = −1; prefix = max(3, 3−4) = 3; suffix = max(−4, −4+3) = −1; best = max(3, −4, 3+(−4)) = 3	(−1, 3, −1, 3)
5	combine (0,3)	total = 0; prefix = max(2, 1+3) = 4; suffix = max(−1, −1+1) = 0; best = max(2, 3, 1+3) = 4	(0, 4, 0, **4**)

Answer 4 = 2 + (−1) + 3 ✓ (the optimal block crosses the middle).

Stage 4 – Validation

Student A – Mathematical validation

Recurrence: $T(n) = 2T(n/2) + c, \quad T(1) = d$

- **2T(n/2):** two recursive calls, each on half the elements (left and right halves).
- **c:** the combine step – four additions and four comparisons, i.e. constant work independent of n (here c = 8 basic operations).

- **d:** the base case (one element, no work beyond returning the tuple).

Solution. The recursion tree has 2^i nodes at depth i, each costing c, for i = 0 … $\log_2 n - 1$, plus n leaves: T(n) = c(1 + 2 + … + n/2) + dn = c(n − 1) + dn = Θ(n). By the Master Theorem a = 2, b = 2, f(n) = c = $\Theta(n^0)$: n^($\log_2 2$) = n dominates f(n), Case 1 → **T(n) = Θ(n)**, with O(log n) recursion depth. (Student A's first design has f(n) = Θ(n) and gives Θ(n log n) by Case 2; the 4-tuple makes the combine O(1) and removes the log factor.) Brute force is $\Theta(n^2)$.

Student B – Experimental validation

Correctness tests (design). single element, single negative, all negative (the answer must be the largest element, not 0), all positive (answer = total), mixed values with the optimum inside one half, crossing the midpoint, and zeros; additionally 2,000 random arrays (n = 1…30, values −20…20) were compared automatically with the $O(n^2)$ brute force, Kadane's O(n) algorithm and the scan-based first design – all outputs agreed.

Table 7.1 – Correctness test cases

Test case	Array	D&C result	Brute force	Kadane
single element	[5]	5	5	5
single negative	[-3]	-3	-3	-3
all negative	[-4, -2, -7, -1, -9]	-1	-1	-1
all positive	[3, 1, 4, 1, 5]	14	14	14
mixed (CLRS-style)	[13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]	43	43	43
crossing optimum	[-2, 5, 6, -1, 7, -9, 3]	17	17	17
with zeros	[0, -1, 0, 0, -2]	0	0	0

Table 7.2 – Basic operation counts (additions + comparisons) and running time

n	Final D&C (4-tuple)	Scan-based design (Student A)	Brute force $O(n^2)$	D&C time	Brute-force time
16	120	128	272	14 µs	17 µs
64	504	768	4,160	57 µs	196 µs
256	2,040	4,096	65,792	233 µs	3.15 ms

n	Final D&C (4-tuple)	Scan-based design (Student A)	Brute force $O(n^2)$	D&C time	Brute-force time
1,024	8,184	20,480	1,049,600	1.02 ms	58.49 ms
4,096	32,760	98,304	16,781,312	4.23 ms	1.29 s
16,384	131,064	458,752	not run	17.85 ms	not run

Evidence. The final design uses exactly 8(n−1) operations for n = 2^k (120, 504, 2,040, 8,184, … for n = 16, 64, 256, 1024), i.e. linear growth that matches Θ(n): quadrupling n multiplies the count by 4. The scan-based first design grows as $2n \cdot \log_2 n$ (20,480 at n = 1024, 458,752 at n = 16,384 – 3.5× the final design), supporting the decision to retain B's idea of richer return values. Brute force performs about n^2 operations (16.8 million at n = 4096 against 32,760 for D&C) and takes 1.29 s versus 4.2 ms. Kadane's algorithm (O(n), O(1) memory) gave identical answers and serves as an independent oracle.

Stage 5 – Cross-defence

Student A – what experimental evidence supports the group's design? (1) Correctness: all 7 designed cases and 2,000 random arrays match brute force and Kadane, including the all-negative case that breaks naive "max with 0" thinking. (2) Complexity: operation counts equal 8(n−1) exactly for n = 2^k, so doubling n doubles the work, as predicted by Θ(n). (3) Improvement over the baselines: 4,096 → brute force needs 16.8 million operations vs. 32,760, and the 4-tuple design needs about 3× fewer operations than the first scan-based design at n = 16,384 and is asymptotically better.

Student B – how does the recurrence arise from the decomposition? The array is split into two halves, each solved by one recursive call on n/2 elements – this gives the term 2T(n/2). The four values returned by each call allow the parent to build its own tuple using a fixed number of additions and comparisons (the combine formulas) – this gives the constant c. The recursion stops at single elements – T(1) = d. Hence T(n) = 2T(n/2) + c, which solves to Θ(n).

Stage 6 – Joint evaluation

- **Why the problem is suitable for D&C:** the problem splits into two independent halves of the same kind; every solution either lies inside one half or crosses the boundary, and the crossing case is determined by small summaries (best suffix of the left, best prefix of the right) that can be maintained recursively.

- **Computational work reduced:** the $O(n^2)$ enumeration of all subarrays is replaced by a $\Theta(n)$ computation; the work shared between overlapping subarrays is computed once per node and reused through the tuple.
- **Additional costs introduced by recursion:** $O(\log n)$ stack space, function-call overhead and creation of a tuple at every node ($2n - 1$ calls for n elements); this is why D&C is not faster than a simple loop for small n (at $n = 16$ the times are almost equal).
- **When D&C might not be appropriate:** when a simpler algorithm with the same order exists and is easier to run (Kadane's one-pass method is also $\Theta(n)$ with $O(1)$ extra memory and no recursion); when the data arrives as a stream; when subproblems overlap heavily (dynamic programming is better); when the combine step is as costly as solving the whole problem (as in our scan-based first design, which costs $\Theta(n)$ per level); and for tiny inputs where recursion overhead dominates. D&C does, however, parallelise naturally (the two halves can run on separate processors) and fits segment-tree style range queries, which a sequential Kadane scan does not.

Conclusion – what should an algorithm designer look for before choosing Divide and Conquer?

1. A problem that can be broken into **smaller instances of the same problem** (self-similarity) with a natural, cheap way to split.
2. **Independent subproblems** (little or no overlap); otherwise dynamic programming or memoisation is more suitable.
3. A **small, well-defined base case** and a way to state exactly what each subproblem must return.
4. A **combine step cheaper than the work saved** – ideally $O(1)$ or $O(n)$ so the recurrence has the form $aT(n/b) + f(n)$ with a favourable Master-Theorem case; the split should be balanced.
5. A real **improvement over the straightforward baseline** (confirmed by recurrence analysis and by experiment), considering recursion overhead, extra memory and a cut-off to a simple method for small inputs.

Contribution Record

This table follows the primary responsibilities defined in the assignment. Edit it so that it reflects the work the pair actually did before submission.

Question	**Student A – Primary contribution**	**Student B – Primary contribution**	**Joint work**
Q1	Part A: decomposition design and mathematical analysis. Part B: experimental investigation (probe counts).	Part A: experimental validation. Part B: elimination logic and efficiency argument.	Strategy, base case, implementation and testing of both algorithms
Q2	Quick Sort investigation: balanced/unbalanced inputs, partition sizes, recursion depth, pivot influence	Comparative D&C analysis: Merge Sort vs Quick Sort, recursion-tree height, $O(n^2)$ argument	Implementations, pivot strategy, experiments, joint design question
Q3	Algorithmic derivation: decomposition, recurrence, asymptotic complexity	Crossover investigation: input sizes, timings, overhead analysis	Both implementations, crossover experiment, joint conclusion
Q4	Experimental and engineering analysis: sizes, counts, timings, crossover, overheads	Algebraic design and correctness: block decomposition, 8→7 transformation, padding	Study, implementations, correctness verification, engineering decision
Q5	Decomposition design: organisation, division, return values, combine, boundary strip	Comparative evidence: datasets, distance counts, growth, practical crossover	Review of brute force, D&C design and implementation, comparison
Q6	Median of Medians investigation; verification answer on Meet-in-the-Middle	Meet-in-the-Middle investigation; verification answer on pivot quality	Cross-understanding of both investigations

Question	Student A – Primary contribution	Student B – Primary contribution	Joint work
Q7	Independent proposal A; mathematical validation (recurrence)	Independent proposal B; experimental validation (tests, counts)	Stage 2 comparison, Stage 3 final design, Stage 5 cross-defence, Stage 6 evaluation

Primary contribution does not imply exclusive responsibility; both students remain accountable for the complete assignment and are prepared for the individual viva (primary question, partner's question, transfer question).

Appendix A – Pseudocode Quality Checklist (DAA Handbook §17)

Check	Status
Inputs and outputs clearly defined	✓
Variables/states have clear meaning	✓
Required variables initialised	✓
Processing order unambiguous; decision conditions explicit	✓
Loops/recursion have clear termination; base cases addressed (shown for every recursive algorithm)	✓
Returned result clearly identified	✓
Programming-language syntax avoided (← assignment, IF/FOR/WHILE … END)	✓
Algorithm can be manually traced (traces given for MaxMin, BinarySearch, MaxSubarray)	✓
Another programmer could implement it without guessing	✓

Appendix B – Implementation Source Code (Python, instrumented)

All results in the tables above were produced by this code. Counters (c[0], c[1]) record comparisons, multiplications, additions or distance computations. Timings were measured with time.perf_counter (best of several repetitions) on a single machine; absolute times will differ on other computers, but the counts and the trends will not.

```
"""DAA Assignment CO3 - Divide and Conquer implementations (instrumented)."""
import sys, math, random
sys.setrecursionlimit(100000)

# ---------------- Q1 : Max/Min and Binary Search ----------------
def maxmin(A, lo, hi, c):
    """Divide-and-conquer (tournament) max/min. c[0] counts element comparisons."""
    if lo == hi:
        return A[lo], A[lo]
    if hi == lo + 1:
        c[0] += 1
        return (A[lo], A[hi]) if A[lo] < A[hi] else (A[hi], A[lo])
    mid = (lo + hi) // 2
    mn1, mx1 = maxmin(A, lo, mid, c)
    mn2, mx2 = maxmin(A, mid + 1, hi, c)
    c[0] += 2
    return (mn1 if mn1 < mn2 else mn2), (mx1 if mx1 > mx2 else mx2)

def maxmin_naive(A, c):
    mn = mx = A[0]
    for x in A[1:]:
        c[0] += 1
        if x < mn: mn = x
        c[0] += 1
        if x > mx: mx = x
    return mn, mx

def binary_search(A, target, c):
    """Returns index or -1. c[0] counts probes (each probe = one 3-way decision on A[mid])."""
```

```
    low, high = 0, len(A) - 1
    while low <= high:
        mid = (low + high) // 2
        c[0] += 1
        if A[mid] == target: return mid
        elif target < A[mid]: high = mid - 1
        else: low = mid + 1
    return -1

# ---------------- Q2 : Merge Sort and Quick Sort ----------------
def merge_sort(A, c, d=1, st=None):
    if st is not None and d > st['depth']: st['depth'] = d
    if len(A) <= 1: return A
    mid = len(A) // 2
    L = merge_sort(A[:mid], c, d + 1, st); R = merge_sort(A[mid:], c, d + 1, st)
    out = []; i = j = 0
    while i < len(L) and j < len(R):
        c[0] += 1
        if L[i] <= R[j]: out.append(L[i]); i += 1
        else: out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out

def quick_sort(A, c, st, d=1):
    """Pivot strategy: FIRST element. Partition keeps order (less / not-less lists)."""
    if d > st['depth']: st['depth'] = d
    if len(A) <= 1: return A
    p = A[0]; L = []; R = []
    for x in A[1:]:
        c[0] += 1
        if x < p: L.append(x)
        else: R.append(x)
    st['splits'].append((d, len(L), len(R)))
    return quick_sort(L, c, st, d + 1) + [p] + quick_sort(R, c, st, d + 1)
```

```
def balanced_input(vals):
    """Permutation of sorted list 'vals' for which first-element pivot is always the median."""
    if not vals: return []
    m = len(vals) // 2
    return [vals[m]] + balanced_input(vals[:m]) + balanced_input(vals[m + 1:])

# ---------- Q3 : Karatsuba ----------
def standard_mul(x, y, c):
    a = [int(ch) for ch in str(x)][::-1]; b = [int(ch) for ch in str(y)][::-1]
    res = [0] * (len(a) + len(b))
    for i, ai in enumerate(a):
        for j, bj in enumerate(b):
            c[0] += 1
            res[i + j] += ai * bj
    carry = 0
    for k in range(len(res)):
        t = res[k] + carry; res[k] = t % 10; carry = t // 10
    return int(''.join(map(str, res[::-1])))

def karatsuba(x, y, c):
    if x < 10 or y < 10:
        c[0] += len(str(x)) * len(str(y))     # single-digit products needed
        return x * y
    n = max(len(str(x)), len(str(y))); m = n // 2; p = 10 ** m
    a, b = divmod(x, p); cc, d = divmod(y, p)
    z2 = karatsuba(a, cc, c); z0 = karatsuba(b, d, c)
    z1 = karatsuba(a + b, cc + d, c) - z2 - z0
    return z2 * p * p + z1 * p + z0

# ---------- Q4 : Strassen ----------
def m_add(A, B, c=None):
    n = len(A)
    if c is not None: c[1] += n * n
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
def m_sub(A, B, c=None):
```

```
    n = len(A)
    if c is not None: c[1] += n * n
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def conv_mul(A, B, c=None):
    n = len(A); Bt = list(zip(*B))
    if c is not None: c[0] += n ** 3; c[1] += n * n * (n - 1)
    return [[sum(a * b for a, b in zip(row, col)) for col in Bt] for row in A]

def strassen(A, B, c=None, cutoff=1):
    """c[0] = scalar multiplications, c[1] = scalar additions/subtractions."""
    n = len(A)
    if n <= cutoff:
        return conv_mul(A, B, c)
    h = n // 2
    q = lambda M, r, s: [row[s:s + h] for row in M[r:r + h]]
    A11, A12, A21, A22 = q(A, 0, 0), q(A, 0, h), q(A, h, 0), q(A, h, h)
    B11, B12, B21, B22 = q(B, 0, 0), q(B, 0, h), q(B, h, 0), q(B, h, h)
    M1 = strassen(m_add(A11, A22, c), m_add(B11, B22, c), c, cutoff)
    M2 = strassen(m_add(A21, A22, c), B11, c, cutoff)
    M3 = strassen(A11, m_sub(B12, B22, c), c, cutoff)
    M4 = strassen(A22, m_sub(B21, B11, c), c, cutoff)
    M5 = strassen(m_add(A11, A12, c), B22, c, cutoff)
    M6 = strassen(m_sub(A21, A11, c), m_add(B11, B12, c), c, cutoff)
    M7 = strassen(m_sub(A12, A22, c), m_add(B21, B22, c), c, cutoff)
    C11 = m_add(m_sub(m_add(M1, M4, c), M5, c), M7, c)
    C12 = m_add(M3, M5, c)
    C21 = m_add(M2, M4, c)
    C22 = m_add(m_sub(m_add(M1, M3, c), M2, c), M6, c)
    return [C11[i] + C12[i] for i in range(h)] + [C21[i] + C22[i] for i in range(h)]

def pad_to_pow2(M):
    n = len(M); s = 1
    while s < n: s *= 2
    return [row + [0] * (s - n) for row in M] + [[0] * s for _ in range(s - n)]
```

```
def strassen_any(A, B, c=None, cutoff=1):
    n = len(A)
    C = strassen(pad_to_pow2(A), pad_to_pow2(B), c, cutoff)
    return [row[:n] for row in C[:n]]

# ---------------- Q5 : Closest pair ----------------
def dist(p, q, c):
    c[0] += 1
    return math.hypot(p[0] - q[0], p[1] - q[1])

def closest_brute(P, c):
    best = float('inf')
    for i in range(len(P)):
        for j in range(i + 1, len(P)):
            d = dist(P[i], P[j], c)
            if d < best: best = d
    return best

def closest_dc(P, c):
    Px = sorted(P); Py = sorted(P, key=lambda p: (p[1], p[0]))
    return _rec(Px, Py, c)

def _rec(Px, Py, c):
    n = len(Px)
    if n <= 3: return closest_brute(Px, c)
    mid = n // 2; midx = Px[mid][0]
    Pl, Pr = Px[:mid], Px[mid:]
    left = set(Pl)
    Yl = [p for p in Py if p in left]; Yr = [p for p in Py if p not in left]
    d = min(_rec(Pl, Yl, c), _rec(Pr, Yr, c))
    strip = [p for p in Py if abs(p[0] - midx) < d]
    for i in range(len(strip)):
        j = i + 1
        while j < len(strip) and strip[j][1] - strip[i][1] < d:
```

```
            d = min(d, dist(strip[i], strip[j], c)); j += 1
    return d

# ---------- Q6 : Selection and Meet-in-the-Middle ----------
def _isort(g, c):
    g = g[:]
    for i in range(1, len(g)):
        k = g[i]; j = i - 1
        while j >= 0:
            c[0] += 1
            if g[j] > k: g[j + 1] = g[j]; j -= 1
            else: break
        g[j + 1] = k
    return g

def mom_select(A, k, c, st=None):
    """k-th smallest (0-indexed), groups of 5."""
    n = len(A)
    if n <= 5: return _isort(A, c)[k]
    meds = [_isort(A[i:i + 5], c)[len(A[i:i + 5]) // 2] for i in range(0, n, 5)]
    m = mom_select(meds, (len(meds) - 1) // 2, c)
    L = [x for x in A if x < m]; R = [x for x in A if x > m]
    c[0] += n - 1
    if st is not None: st.append(max(len(L), len(R)) / n)
    if k < len(L): return mom_select(L, k, c, st)
    if k == len(L): return m
    return mom_select(R, k - len(L) - 1, c, st)

def quickselect(A, k, c, pivot='first', st=None):
    n = len(A)
    if n == 1: return A[0]
    p = A[0] if pivot == 'first' else random.choice(A)
    L = [x for x in A if x < p]; R = [x for x in A if x > p]
    c[0] += n - 1
    if st is not None: st.append(max(len(L), len(R)) / n)
```

```
    if k < len(L): return quickselect(L, k, c, pivot, st)
    if k == len(L): return p
    return quickselect(R, k - len(L) - 1, c, pivot, st)

def subset_sums(items):
    sums = [0]
    for x in items: sums += [s + x for s in sums]
    return sums

def mitm_subset_sum(items, target, c):
    """c[0]=sums generated, c[1]=search steps. Returns True/False."""
    h = len(items) // 2
    L = subset_sums(items[:h]); R = subset_sums(items[h:]); c[0] += len(L) + len(R)
    L.sort(); R.sort()
    i, j = 0, len(R) - 1
    while i < len(L) and j >= 0:
        c[1] += 1
        s = L[i] + R[j]
        if s == target: return True
        if s < target: i += 1
        else: j -= 1
    return False

def brute_subset_sum(items, target, c):
    n = len(items)
    for mask in range(1 << n):
        c[0] += 1
        s = 0
        for i in range(n):
            if mask >> i & 1: s += items[i]
        if s == target: return True
    return False

# ---------- Q7 : Maximum Subarray (design challenge) ----------
def mss_dc(A, lo, hi, c):
```

```
    """Returns (total, best_prefix, best_suffix, best_sub). c[0] = basic ops (add+compare)."""
    if lo == hi:
        return A[lo], A[lo], A[lo], A[lo]
    mid = (lo + hi) // 2
    a = mss_dc(A, lo, mid, c); b = mss_dc(A, mid + 1, hi, c)
    c[0] += 8                   # 4 additions + 4 comparisons in COMBINE
    total = a[0] + b[0]
    pre = max(a[1], a[0] + b[1]); suf = max(b[2], b[0] + a[2])
    best = max(a[3], b[3], a[2] + b[1])
    return total, pre, suf, best

def mss_scan(A, lo, hi, c):
    """Student A's first design: split, then SCAN outward from the middle to find crossing sum."""
    if lo == hi: return A[lo]
    mid = (lo + hi) // 2
    l = mss_scan(A, lo, mid, c); r = mss_scan(A, mid + 1, hi, c)
    s = 0; bl = -float('inf')
    for i in range(mid, lo - 1, -1):
        s += A[i]; c[0] += 2
        if s > bl: bl = s
    s = 0; br = -float('inf')
    for i in range(mid + 1, hi + 1):
        s += A[i]; c[0] += 2
        if s > br: br = s
    return max(l, r, bl + br)

def mss_brute(A, c):
    best = -float('inf')
    for i in range(len(A)):
        s = 0
        for j in range(i, len(A)):
            s += A[j]; c[0] += 2
            if s > best: best = s
    return best
```

```
def kadane(A):
    best = cur = A[0]
    for x in A[1:]:
        cur = max(x, cur + x); best = max(best, cur)
    return best
```

Appendix C – Python Code and Execution Output (Screenshots)

For each question the Python code is shown first, followed by the output produced when the program was run under Python 3 (a small driver program calls the functions and prints the results; exit code 0 for every program). The experiment tables in the main text were generated by the same code.

Q1 – Max/Min and Binary Search

Code – Q1

main.py (Q1: part 1 of 1)

```python
"""DAA Assignment CO3 - Divide and Conquer implementations (instrumented)."""
import sys, math, random
sys.setrecursionlimit(100000)

# ---------------- Q1 : Max/Min and Binary Search ----------------
def maxmin(A, lo, hi, c):
    """Divide-and-conquer (tournament) max/min. c[0] counts element comparisons."""
    if lo == hi:
        return A[lo], A[lo]
    if hi == lo + 1:
        c[0] += 1
        return (A[lo], A[hi]) if A[lo] < A[hi] else (A[hi], A[lo])
    mid = (lo + hi) // 2
    mn1, mx1 = maxmin(A, lo, mid, c)
    mn2, mx2 = maxmin(A, mid + 1, hi, c)
    c[0] += 2
    return (mn1 if mn1 < mn2 else mn2), (mx1 if mx1 > mx2 else mx2)

def maxmin_naive(A, c):
    mn = mx = A[0]
    for x in A[1:]:
        c[0] += 1
        if x < mn: mn = x
        c[0] += 1
        if x > mx: mx = x
    return mn, mx

def binary_search(A, target, c):
    """Returns index or -1. c[0] counts probes (each probe = one 3-way decision on A[mid])."""
    low, high = 0, len(A) - 1
    while low <= high:
        mid = (low + high) // 2
        c[0] += 1
        if A[mid] == target: return mid
        elif target < A[mid]: high = mid - 1
        else: low = mid + 1
    return -1
```

Output – Q1

```
Output

$ python3 demo_q1.py
A = [18, 73, 98, 9, 33, 16, 64, 58, 61, 84, 49, 27, 13, 63, 4, 50]
MaxMin  -> min = 4  max = 98  comparisons = 22 (3n/2-2 = 22)
Independent search comparisons = 30 (2(n-1) = 30)
Sorted A = [4, 9, 13, 16, 18, 27, 33, 49, 50, 58, 61, 63, 64, 73, 84, 98]
BinarySearch target 63 -> index 11  probes = 2
BinarySearch target 1000 -> index -1  probes = 5

[Program finished with exit code 0]
```

Q2 – Merge Sort and Quick Sort

Code – Q2

main.py (Q2: part 1 of 1)

```python
# ---------------- Q2 : Merge Sort and Quick Sort ----------------
def merge_sort(A, c, d=1, st=None):
    if st is not None and d > st['depth']: st['depth'] = d
    if len(A) <= 1: return A
    mid = len(A) // 2
    L = merge_sort(A[:mid], c, d + 1, st); R = merge_sort(A[mid:], c, d + 1, st)
    out = []; i = j = 0
    while i < len(L) and j < len(R):
        c[0] += 1
        if L[i] <= R[j]: out.append(L[i]); i += 1
        else: out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out

def quick_sort(A, c, st, d=1):
    """Pivot strategy: FIRST element. Partition keeps order (less / not-less lists)."""
    if d > st['depth']: st['depth'] = d
    if len(A) <= 1: return A
    p = A[0]; L = []; R = []
    for x in A[1:]:
        c[0] += 1
        if x < p: L.append(x)
        else: R.append(x)
    st['splits'].append((d, len(L), len(R)))
    return quick_sort(L, c, st, d + 1) + [p] + quick_sort(R, c, st, d + 1)

def balanced_input(vals):
    """Permutation of sorted list 'vals' for which first-element pivot is always the median."""
    if not vals: return []
    m = len(vals) // 2
    return [vals[m]] + balanced_input(vals[:m]) + balanced_input(vals[m + 1:])
```

Output – Q2

```
Output

$ python3 demo_q2.py
n = 1023
input      QS compares   QS depth  MS compares   MS depth
balanced          8194         10         6990         11
random           13835         25         8943         11
sorted          522753       1023         5110         11

[Program finished with exit code 0]
```

Q3 – Karatsuba multiplication

Code – Q3

main.py (Q3: part 1 of 1)

```
# ---------------- Q3 : Karatsuba ----------------
def standard_mul(x, y, c):
    a = [int(ch) for ch in str(x)][::-1]; b = [int(ch) for ch in str(y)][::-1]
    res = [0] * (len(a) + len(b))
    for i, ai in enumerate(a):
        for j, bj in enumerate(b):
            c[0] += 1
            res[i + j] += ai * bj
    carry = 0
    for k in range(len(res)):
        t = res[k] + carry; res[k] = t % 10; carry = t // 10
    return int(''.join(map(str, res[::-1])))

def karatsuba(x, y, c):
    if x < 10 or y < 10:
        c[0] += len(str(x)) * len(str(y))     # single-digit products needed
        return x * y
    n = max(len(str(x)), len(str(y))); m = n // 2; p = 10 ** m
    a, b = divmod(x, p); cc, d = divmod(y, p)
    z2 = karatsuba(a, cc, c); z0 = karatsuba(b, d, c)
    z1 = karatsuba(a + b, cc + d, c) - z2 - z0
    return z2 * p * p + z1 * p + z0
```

Output – Q3

```
Output

$ python3 demo_q3.py
Karatsuba(1234, 5678) = 7006652  (expected 7006652 )
n =  16 digits: standard mults =    256, Karatsuba mults =    130
n =  64 digits: standard mults =   4096, Karatsuba mults =   1203
n = 256 digits: standard mults =  65536, Karatsuba mults =  10870

[Program finished with exit code 0]
```

Q4 – Strassen matrix multiplication

Code – Q4 (part 1)

main.py (Q4: part 1 of 2)

```python
# ---------------- Q4 : Strassen ----------------
def m_add(A, B, c=None):
    n = len(A)
    if c is not None: c[1] += n * n
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
def m_sub(A, B, c=None):
    n = len(A)
    if c is not None: c[1] += n * n
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def conv_mul(A, B, c=None):
    n = len(A); Bt = list(zip(*B))
    if c is not None: c[0] += n ** 3; c[1] += n * n * (n - 1)
    return [[sum(a * b for a, b in zip(row, col)) for col in Bt] for row in A]

def strassen(A, B, c=None, cutoff=1):
    """c[0] = scalar multiplications, c[1] = scalar additions/subtractions."""
    n = len(A)
    if n <= cutoff:
        return conv_mul(A, B, c)
    h = n // 2
    q = lambda M, r, s: [row[s:s + h] for row in M[r:r + h]]
    A11, A12, A21, A22 = q(A, 0, 0), q(A, 0, h), q(A, h, 0), q(A, h, h)
    B11, B12, B21, B22 = q(B, 0, 0), q(B, 0, h), q(B, h, 0), q(B, h, h)
    M1 = strassen(m_add(A11, A22, c), m_add(B11, B22, c), c, cutoff)
    M2 = strassen(m_add(A21, A22, c), B11, c, cutoff)
    M3 = strassen(A11, m_sub(B12, B22, c), c, cutoff)
    M4 = strassen(A22, m_sub(B21, B11, c), c, cutoff)
    M5 = strassen(m_add(A11, A12, c), B22, c, cutoff)
    M6 = strassen(m_sub(A21, A11, c), m_add(B11, B12, c), c, cutoff)
    M7 = strassen(m_sub(A12, A22, c), m_add(B21, B22, c), c, cutoff)
    C11 = m_add(m_sub(m_add(M1, M4, c), M5, c), M7, c)
    C12 = m_add(M3, M5, c)
    C21 = m_add(M2, M4, c)
    C22 = m_add(m_sub(m_add(M1, M3, c), M2, c), M6, c)
    return [C11[i] + C12[i] for i in range(h)] + [C21[i] + C22[i] for i in range(h)]

def pad_to_pow2(M):
```

Code – Q4 (part 2)

main.py (Q4: part 2 of 2)

```python
    n = len(M); s = 1
    while s < n: s *= 2
    return [row + [0] * (s - n) for row in M] + [[0] * s for _ in range(s - n)]

def strassen_any(A, B, c=None, cutoff=1):
    n = len(A)
    C = strassen(pad_to_pow2(A), pad_to_pow2(B), c, cutoff)
    return [row[:n] for row in C[:n]]
```

Output – Q4

```
Output

$ python3 demo_q4.py
n = 16  results equal: True
conventional: mults = 4096  adds = 3840
Strassen    : mults = 2401  adds = 12870
n = 10 (padded to 16) results equal: True

[Program finished with exit code 0]
```

Q5 – Closest pair

Code – Q5

main.py (Q5: part 1 of 1)

```python
# ---------------- Q5 : Closest pair ----------------
def dist(p, q, c):
    c[0] += 1
    return math.hypot(p[0] - q[0], p[1] - q[1])

def closest_brute(P, c):
    best = float('inf')
    for i in range(len(P)):
        for j in range(i + 1, len(P)):
            d = dist(P[i], P[j], c)
            if d < best: best = d
    return best

def closest_dc(P, c):
    Px = sorted(P); Py = sorted(P, key=lambda p: (p[1], p[0]))
    return _rec(Px, Py, c)

def _rec(Px, Py, c):
    n = len(Px)
    if n <= 3: return closest_brute(Px, c)
    mid = n // 2; midx = Px[mid][0]
    Pl, Pr = Px[:mid], Px[mid:]
    left = set(Pl)
    Yl = [p for p in Py if p in left]; Yr = [p for p in Py if p not in left]
    d = min(_rec(Pl, Yl, c), _rec(Pr, Yr, c))
    strip = [p for p in Py if abs(p[0] - midx) < d]
    for i in range(len(strip)):
        j = i + 1
        while j < len(strip) and strip[j][1] - strip[i][1] < d:
            d = min(d, dist(strip[i], strip[j], c)); j += 1
    return d
```

Output – Q5

```
Output

$ python3 demo_q5.py
points = 1000
brute force  : d = 416.2932  distance calcs = 499500
divide&conquer: d = 416.2932  distance calcs = 1096

[Program finished with exit code 0]
```

Q6 – Median of Medians and Meet-in-the-Middle

Code – Q6 (part 1)

main.py (Q6: part 1 of 2)

```
173 # ---------------- Q6 : Selection and Meet-in-the-Middle ----------------
174 def _isort(g, c):
175     g = g[:]
176     for i in range(1, len(g)):
177         k = g[i]; j = i - 1
178         while j >= 0:
179             c[0] += 1
180             if g[j] > k: g[j + 1] = g[j]; j -= 1
181             else: break
182         g[j + 1] = k
183     return g
184 
185 def mom_select(A, k, c, st=None):
186     """k-th smallest (0-indexed), groups of 5."""
187     n = len(A)
188     if n <= 5: return _isort(A, c)[k]
189     meds = [_isort(A[i:i + 5], c)[len(A[i:i + 5]) // 2] for i in range(0, n, 5)]
190     m = mom_select(meds, (len(meds) - 1) // 2, c)
191     L = [x for x in A if x < m]; R = [x for x in A if x > m]
192     c[0] += n - 1
193     if st is not None: st.append(max(len(L), len(R)) / n)
194     if k < len(L): return mom_select(L, k, c, st)
195     if k == len(L): return m
196     return mom_select(R, k - len(L) - 1, c, st)
197 
198 def quickselect(A, k, c, pivot='first', st=None):
199     n = len(A)
200     if n == 1: return A[0]
201     p = A[0] if pivot == 'first' else random.choice(A)
202     L = [x for x in A if x < p]; R = [x for x in A if x > p]
203     c[0] += n - 1
204     if st is not None: st.append(max(len(L), len(R)) / n)
205     if k < len(L): return quickselect(L, k, c, pivot, st)
206     if k == len(L): return p
207     return quickselect(R, k - len(L) - 1, c, pivot, st)
208 
209 def subset_sums(items):
210     sums = [0]
```

Code – Q6 (part 2)

main.py (Q6: part 2 of 2)

```
    for x in items: sums += [s + x for s in sums]
    return sums

def mitm_subset_sum(items, target, c):
    """c[0]=sums generated, c[1]=search steps. Returns True/False."""
    h = len(items) // 2
    L = subset_sums(items[:h]); R = subset_sums(items[h:]); c[0] += len(L) + len(R)
    L.sort(); R.sort()
    i, j = 0, len(R) - 1
    while i < len(L) and j >= 0:
        c[1] += 1
        s = L[i] + R[j]
        if s == target: return True
        if s < target: i += 1
        else: j -= 1
    return False

def brute_subset_sum(items, target, c):
    n = len(items)
    for mask in range(1 << n):
        c[0] += 1
        s = 0
        for i in range(n):
            if mask >> i & 1: s += items[i]
        if s == target: return True
    return False
```

Output – Q6

Output

```
$ python3 demo_q6.py
MoM  k-th smallest = 9699  comparisons = 15626
QuickSelect (first pivot, random data) = 9699  comparisons = 9083
QuickSelect (first pivot, SORTED data) comparisons = 1999000
MoM on SORTED data comparisons = 11004
expected 9699
Meet-in-the-Middle n=20, reachable target -> True  sums generated = 2048  search steps = 608
Meet-in-the-Middle n=20, target -1 -> False  (brute force would check 1048576 subsets)

[Program finished with exit code 0]
```

Q7 – Maximum subarray (design challenge)

Code – Q7 (part 1)

main.py (Q7: part 1 of 2)

```
# ---------------- Q7 : Maximum Subarray (design challenge) ----------------
def mss_dc(A, lo, hi, c):
    """Returns (total, best_prefix, best_suffix, best_sub). c[0] = basic ops (add+compare)."""
    if lo == hi:
        return A[lo], A[lo], A[lo], A[lo]
    mid = (lo + hi) // 2
    a = mss_dc(A, lo, mid, c); b = mss_dc(A, mid + 1, hi, c)
    c[0] += 8                       # 4 additions + 4 comparisons in COMBINE
    total = a[0] + b[0]
    pre = max(a[1], a[0] + b[1]); suf = max(b[2], b[0] + a[2])
    best = max(a[3], b[3], a[2] + b[1])
    return total, pre, suf, best

def mss_scan(A, lo, hi, c):
    """Student A's first design: split, then SCAN outward from the middle to find crossing sum."""
    if lo == hi: return A[lo]
    mid = (lo + hi) // 2
    l = mss_scan(A, lo, mid, c); r = mss_scan(A, mid + 1, hi, c)
    s = 0; bl = -float('inf')
    for i in range(mid, lo - 1, -1):
        s += A[i]; c[0] += 2
        if s > bl: bl = s
    s = 0; br = -float('inf')
    for i in range(mid + 1, hi + 1):
        s += A[i]; c[0] += 2
        if s > br: br = s
    return max(l, r, bl + br)

def mss_brute(A, c):
    best = -float('inf')
    for i in range(len(A)):
        s = 0
        for j in range(i, len(A)):
            s += A[j]; c[0] += 2
            if s > best: best = s
    return best

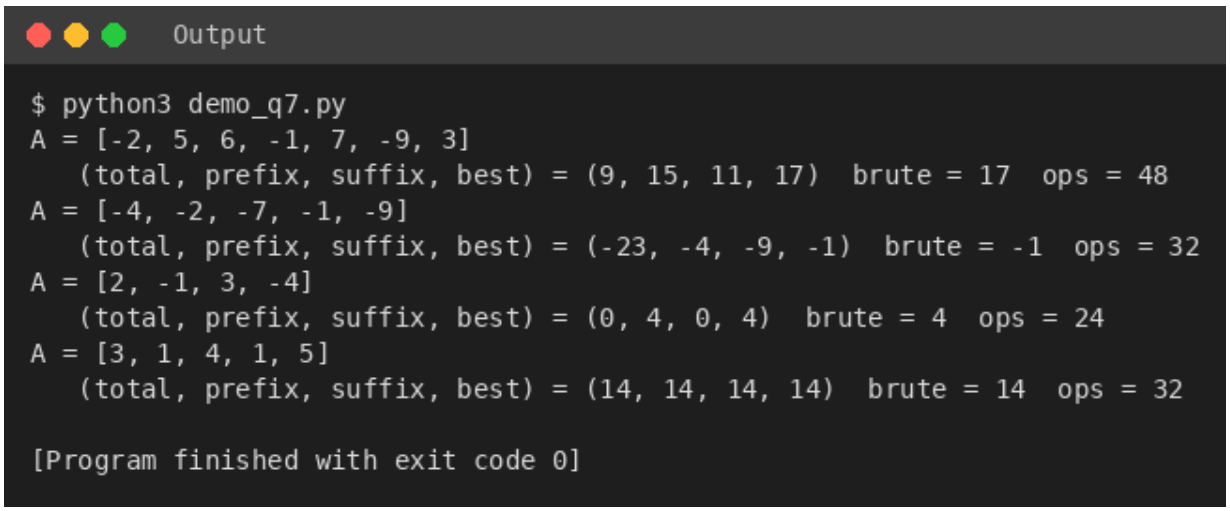
def kadane(A):
```

Code – Q7 (part 2)

main.py (Q7: part 2 of 2)

```
    best = cur = A[0]
    for x in A[1:]:
        cur = max(x, cur + x); best = max(best, cur)
    return best
```

Output – Q7


Output
$ python3 demo_q7.py
A = [-2, 5, 6, -1, 7, -9, 3]
(total, prefix, suffix, best) = (9, 15, 11, 17) brute = 17 ops = 48
A = [-4, -2, -7, -1, -9]
(total, prefix, suffix, best) = (-23, -4, -9, -1) brute = -1 ops = 32
A = [2, -1, 3, -4]
(total, prefix, suffix, best) = (0, 4, 0, 4) brute = 4 ops = 24
A = [3, 1, 4, 1, 5]
(total, prefix, suffix, best) = (14, 14, 14, 14) brute = 14 ops = 32
[Program finished with exit code 0]